

http://algs4.cs.princeton.edu

5.5 DATA COMPRESSION

- *introduction*
- *run-length coding*
- *Huffman compression*
- *LZW compression*



5.5 Data Compression

- *introduction*
- *run-length coding*
- *Huffman compression*
- *LZW compression*

Data compression

Compression redukton madhësinë e një file-i:

- Për të kursyer hapësirë kur të ruhet.
- Për të kursyer kohë kur të transmetohet.
- Sepse shumica e file-ave kanë shumë përsëritje.

Për çka nevojitet compression?

- Ligji i Moore-it : # i tranzistorëve në një chip dyfishohet çdo 18–24 muaj.
- Ligji i Parkinson-it: Të dhënat shtohen sa të ka hapësirë për të mbuluar.
- Tekst, image, audio, video, …

> *“ Everyday, we create 2.5 quintillion bytes of data—so much that 90% of the data in the world today has been created in the last two years alone. ”* — *IBM report on big data (2011)*

Konceptet e para datojnë qysh moti (1950at), teknologjia më e përsosur është e zhvilluar së fundi.

Aplikimet

Generic file compression.

- Files: GZIP, BZIP, 7z.
- Archivers: PKZIP.
- File systems: NTFS, HFS+, ZFS.



Multimedia.

- Images: GIF, JPEG.
- Sound: MP3.
- Video: MPEG, DivX™, HDTV.







Communication.

- ITU-T T4 Group 3 Fax.
- V.42bis modem.
- Skype.





Databases. Google, Facebook, ...

Google™

facebook

Kompresimi dhe zgjerimi pa humbje

Mesazhi. Binary data B që dojmë të kompresojmë

Kompresimi. Gjeneron një "paraqitje" të kompresuar $C(B)$.

Zgjerimi. Rikonstrukton bitstream-in original B.

uses fewer bits (you hope)



Modeli bazik për data compression

Proporcioni i kompresimit. Bita në $C(B)$/ bita në B.

Raste për të diskutuar

Data compression përdoret qysh moti, gjithandej:

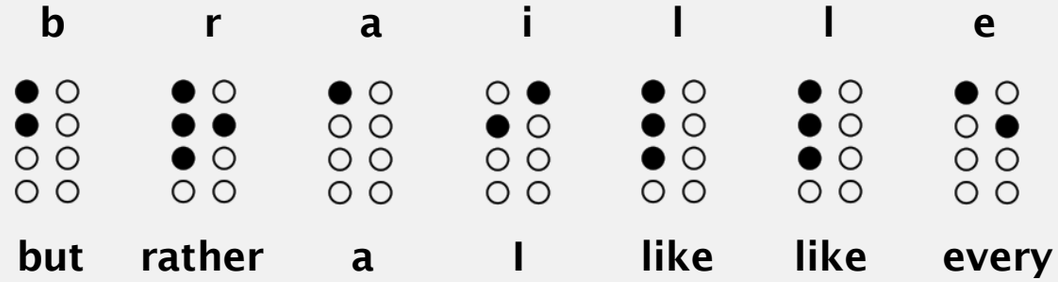
- Sistemet numerike.
- Gjuhet natyrale.
- Notacion matematikor.



$$\sum_{n=1}^{\infty} \frac{1}{n^2} = \frac{\pi^2}{6}$$

Ka lujatur rol kryesorë në zhvillimin e teknologjisë së komunikimit,

- Grade 2 Braille.
- Morse code.
- Sistemet telefonike.



dhe është pjesë e jetës moderne.

- MP3.
- MPEG.



Paraqitja e të dhënave: kodi i gjenomit

Gjenomi. String në alfabetin `{ A, C, T, G }`.

Qëllimi. Të enkodohet një gjenom me N-karaktere: `A T A G A T G C A T A G ...`

ASCII enkodimi standard.

- 8 bits per char.
- $8\,N$ bits.

char	hex	binary
A	41	01000001
C	43	01000011
T	54	01010100
G	47	01000111

Enkodimi 2 bitësh.

- 2 bits per char.
- $2\,N$ bits.

char	binary
A	00
C	01
T	10
G	11

Kodi me gjatësi fikse. k-bit kodi përkrahë alfabet me gjatësi 2^k.

Trivia. Disa databaza gjenome 1990ave kanë përdorë ASCII.

Leximi dhe shkrimi i të dhënave binare

Binary standard input and standard output. Libraritë për read dhe write bits nga standard input dhe në standard output.

public class BinaryStdIn	
boolean readBoolean()	*read 1 bit of data and return as a* `boolean` *value*
char readChar()	*read 8 bits of data and return as a* `char` *value*
char readChar(int r)	*read* `r` *bits of data and return as a* `char` *value*
[*similar methods for* `byte` *(8 bits);* `short` *(16 bits);* `int` *(32 bits);* `long` *and* `double` *(64 bits)*]	
boolean isEmpty()	*is the bitstream empty?*
void close()	*close the bitstream*

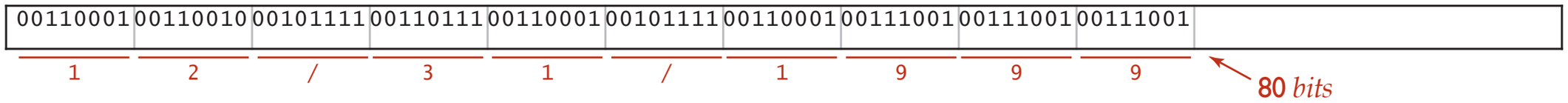
public class BinaryStdOut	
void write(boolean b)	*write the specified bit*
void write(char c)	*write the specified 8-bit* `char`
void write(char c, int r)	*write the* `r` *least significant bits of the specified* `char`
[*similar methods for* `byte` *(8 bits);* `short` *(16 bits);* `int` *(32 bits);* `long` *and* `double` *(64 bits)*]	
void close()	*close the bitstream*

Write i të dhënave binare

Paraqitja e datës. Tri mënyra të ndryshme të paraqitet data 12/31/1999.

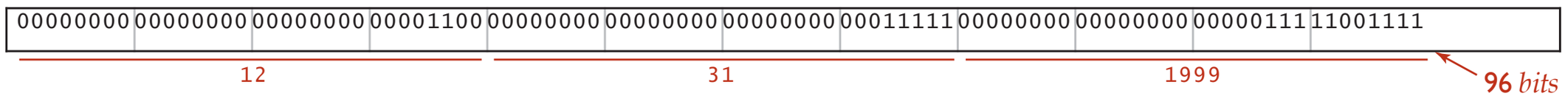
A character stream (StdOut)

```
StdOut.print(month + "/" + day + "/" + year);
```



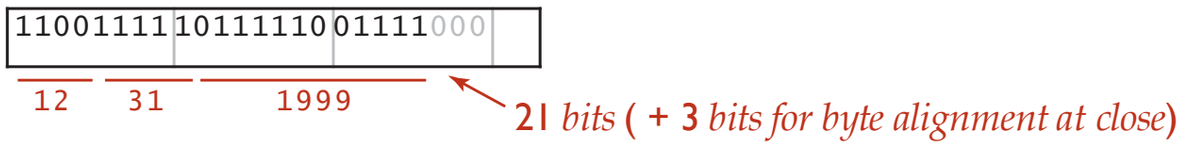
Three ints (BinaryStdOut)

```
BinaryStdOut.write(month);
BinaryStdOut.write(day);
BinaryStdOut.write(year);
```



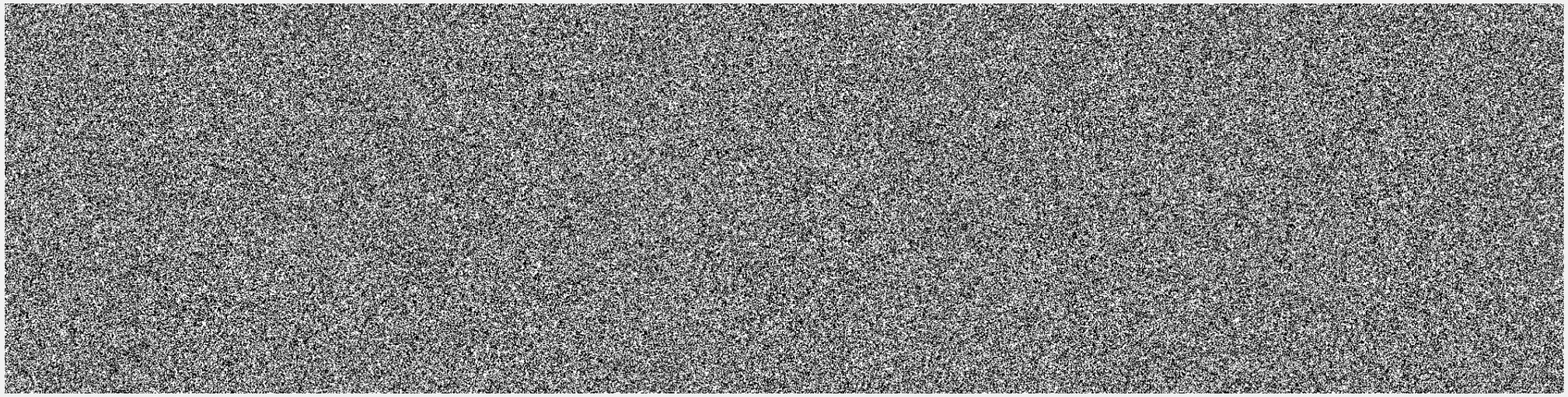
A 4-bit field, a 5-bit field, and a 12-bit field (BinaryStdOut)

```
BinaryStdOut.write(month, 4);
BinaryStdOut.write(day, 5);
BinaryStdOut.write(year, 12);
```



Undecidability

```
% java RandomBits | java PictureDump 2000 500
```



```
1000000 bits
```

A difficult file to compress: one million (pseudo-) random bits

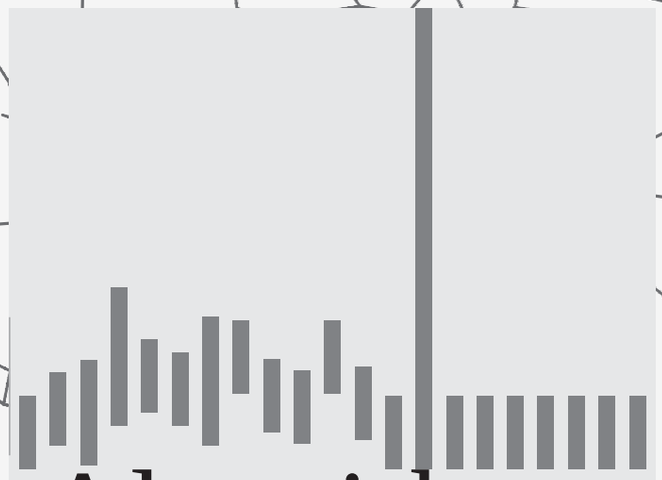
```
public class RandomBits
{
   public static void main(String[] args)
   {
      int x = 11111;
      for (int i = 0; i < 1000000; i++)
      {
         x = x * 314159 + 218281;
         BinaryStdOut.write(x > 0);
      }
      BinaryStdOut.close();
   }
}
```

Rdenudcany in Enlgsih Inagugae

Q. How mcuh rdenudcany is in the Enlgsih lnagugae?

“ ... randomising letters in the middle of words [has] little or no effect on the ability of skilled readers to understand the text. This is easy to denmtrasote. In a pubiltacion of New Scnieitst you could ramdinose all the letetrs, keipeng the first two and last two the same, and reibadailty would hadrly be aftcfeed. My ansaylis did not come to much beucase the thoery at the time was for shape and senqeuce retigcionon. Saberi's work sugsegts we may have some pofrweul palrlael prsooscers at work. The resaon for this is suerly that idnetiyfing coentnt by paarllel prseocsing speeds up regnicoiton. We only need the first and last two letetrs to spot chganes in meniang. ”

— Graham Rawlinson



Robert Sedgewick | Kevin Wayne

http://algs4.cs.princeton.edu

5.5 Data Compression

- *introduction*
- *run-length coding*
- *Huffman compression*
- *LZW compression*

Enkodimi

Lloj i thjeshtë i redundancy në një bitstream. Rradhë e gjatë e bitave që përsëriten.

0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1

40 bits

Paraqitja. Vargje 4-bitëshe që paraqesin rradhët e njëpasnjëshme të 0 dhe 1:

15 0, pastaj 7 1sha, pastaj 7 0, pastaj 11 1sha.

1 1 1 1 0 1 1 1 0 1 1 1 1 0 1 1 ← 16 bits (instead of 40)

15 7 7 11

Aplikimet. JPEG, ITU-T T4 Group 3 Fax, ...

Enkodimi: Java implementimi

```
public class RunLength
{
   private final static int R   = 256;
   private final static int lgR = 8;

   public static void compress()
   {  /* see textbook */  }

   public static void expand()
   {
      boolean bit = false;
      while (!BinaryStdIn.isEmpty())
      {
         int run = BinaryStdIn.readInt(lgR);
         for (int i = 0; i < run; i++)
            BinaryStdOut.write(bit);
         bit = !bit;
      }
      BinaryStdOut.close();
   }

}
```

maximum run-length count

number of bits per count

read 8-bit count from standard input

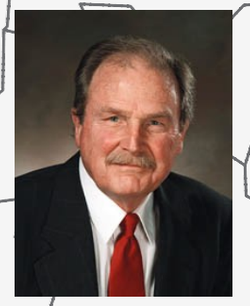
write 1 bit to standard output

pad 0s for byte alignment



5.5 Data Compression

- *introduction*
- *run-length coding*
- *Huffman compression*
- *LZW compression*



David Huffman

Variable-length codes

Përdoret numër i ndryshëm i bitave pë të enkoduar karaktere të ndryshme.

P.sh.. Morse code: •••−−−•••

Mangësi. Paqartësia (dy kuptime të ndryshme)

SOS ?

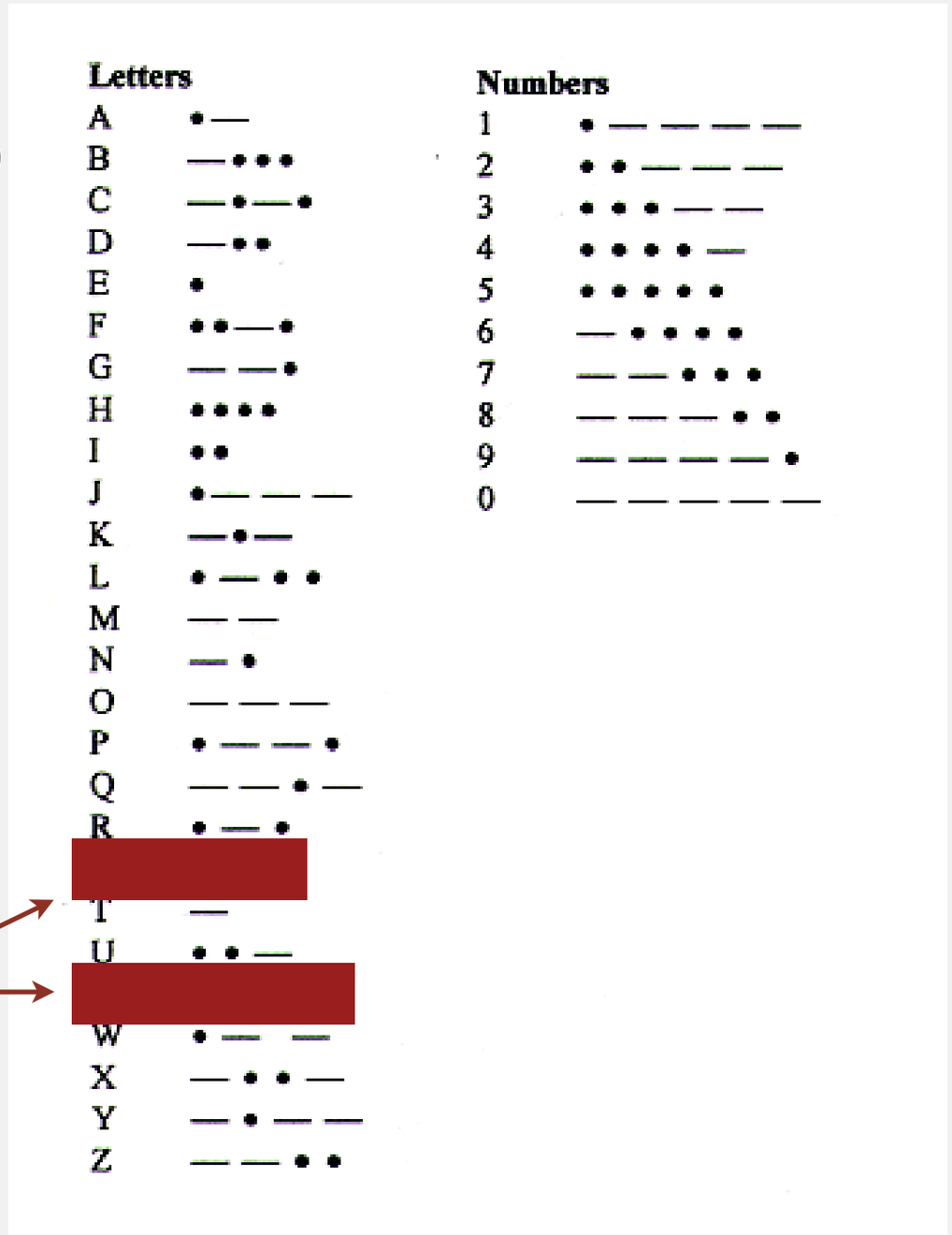
V7 ?

IAMIE ?

EEWNI ?

Në praktikë. Përdoret një zbrastirë që ndanë codewords

codeword for S is a prefix of codeword for V



Variable-length codes

Pyejte. Si të shmanget nga paqartësitë?

Përgjigje. Sigurohet që asnjë codeword nuk është prefix i një codeword tjetër.

Rasti 1. Fixed-length code.

Rasti 2. Shtohet char special për stop tek çdo codewor

Rasti 3. Code prefix-free (që nuk ka prefikse të njejta).

Codeword table

key	*value*
!	101
A	0
B	1111
C	110
D	100
R	1110

Compressed bitstring

01111111001100100011111100101 ←*30 bits*

A B RA CA DA B RA !

Codeword table

key	*value*
!	101
A.	11
B.	00
C	010
D	100
R	011

Compressed bitstring

11000111101011100110001111101 ←*29 bits*

A B R A C A D A B R A !

Prefix-free codes: Paraqitja me trie

Pyetje. Si të paraqitet prefix-free code?

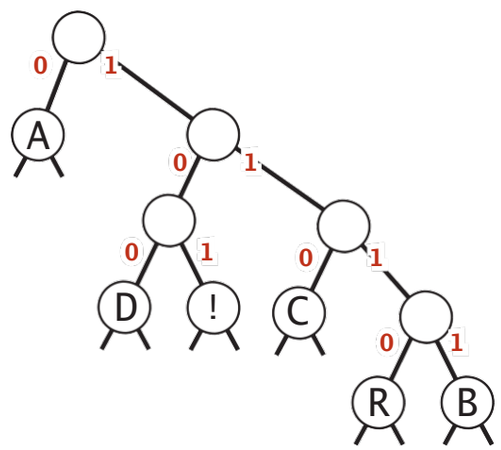
Përgjigje. Me anë të një binary trie!

- Char në leaves.
- Codeword është path prej root deri tek leaves.

Codeword table

key	*value*
!	101
A	0
B	1111
C	110
D	100
R	1110

Trie representation



Compressed bitstring

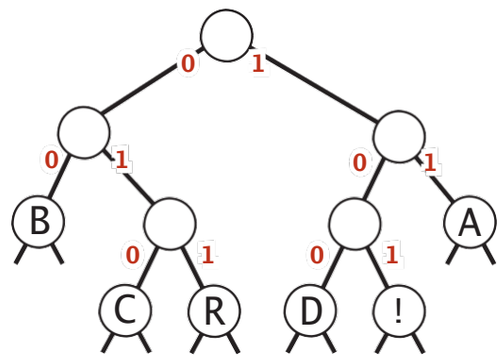
0111111100110010001111111100101 ←30 bits

A B RA CA DA B RA !

Codeword table

key	*value*
!	101
A.	11
B.	00
C	010
D	100
R	011

Trie representation



Compressed bitstring

1100011110101110011000111101 ←29 bits

A B R A C A D A B R A !

Prefix-free code: kompresimi dhe zgjerimi

Kompresioni.

- Metoda 1: start në leaf; ndjeke path deri tek root; print bitat në reverse.
- Metoda 2: krijohet ST (symbol table) të key-value çifteve.

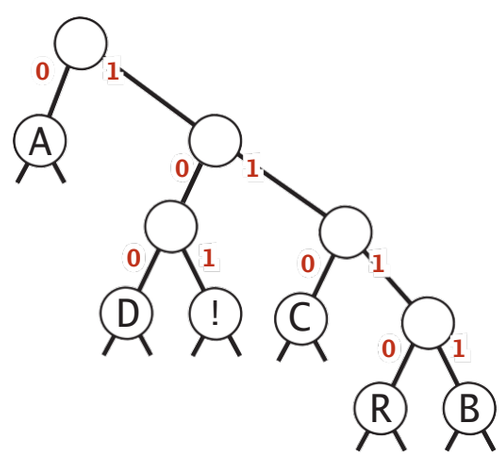
Zgjerimi.

- Startohet në root.
- Shko majtas nëse biti është 0; shko djathtas nëse 1.
- Nëse është leaf node, print char dhe kthehu në root.

Codeword table

key	*value*
!	101
A	0
B	1111
C	110
D	100
R	1110

Trie representation



Compressed bitstring

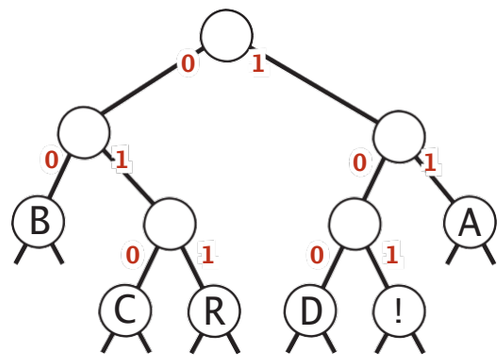
0111111100110010001111111100101 ←*30 bits*

A B RA CA DA B RA !

Codeword table

key	*value*
!	101
A.	11
B.	00
C	010
D	100
R	011

Trie representation



Compressed bitstring

11000111101011100110001111101 ←*29 bits*

A B R A C A D A B R A !

Huffman coding - përmbledhje

Modeli dinamik. Përdoret një prefix-free code për çdo mesazh.

Kompresimi.

- Read mesazhin.
- Ndërtohet prefix-free code më i mirë për mesazh. Si bëhet?
- Write prefix-free code (në formë të trie) në file.
- Kompresohet mesazhi duke përdorprefix-free code.

Zgjerimi.

- Read prefix-free code (në formë të trie) nga file.
- Read mesazhin e kompresuar dhe zgjero duke përdorur trie.

Huffman trie node data type

```
private static class Node implements Comparable<Node>
{
   private final char ch;   // used only for leaf nodes
   private final int freq;  // used only for compress
   private final Node left, right;

   public Node(char ch, int freq, Node left, Node right)
   {
      this.ch    = ch;
      this.freq  = freq;
      this.left  = left;
      this.right = right;
   }

   public boolean isLeaf()
   {  return left == null && right == null; }

   public int compareTo(Node that)
   {  return this.freq - that.freq;  }
}
```

initializing constructor

is Node a leaf?

compare Nodes by frequency (stay tuned)

Prefix-free codes: expansion

```
public void expand()
{
   Node root = readTrie();
   int N = BinaryStdIn.readInt();

   for (int i = 0; i < N; i++)
   {
      Node x = root;
      while (!x.isLeaf())
      {
         if (!BinaryStdIn.readBoolean())
            x = x.left;
         else
            x = x.right;
      }
      BinaryStdOut.write(x.ch, 8);
   }
   BinaryStdOut.close();
}
```

read in encoding trie

read in number of chars

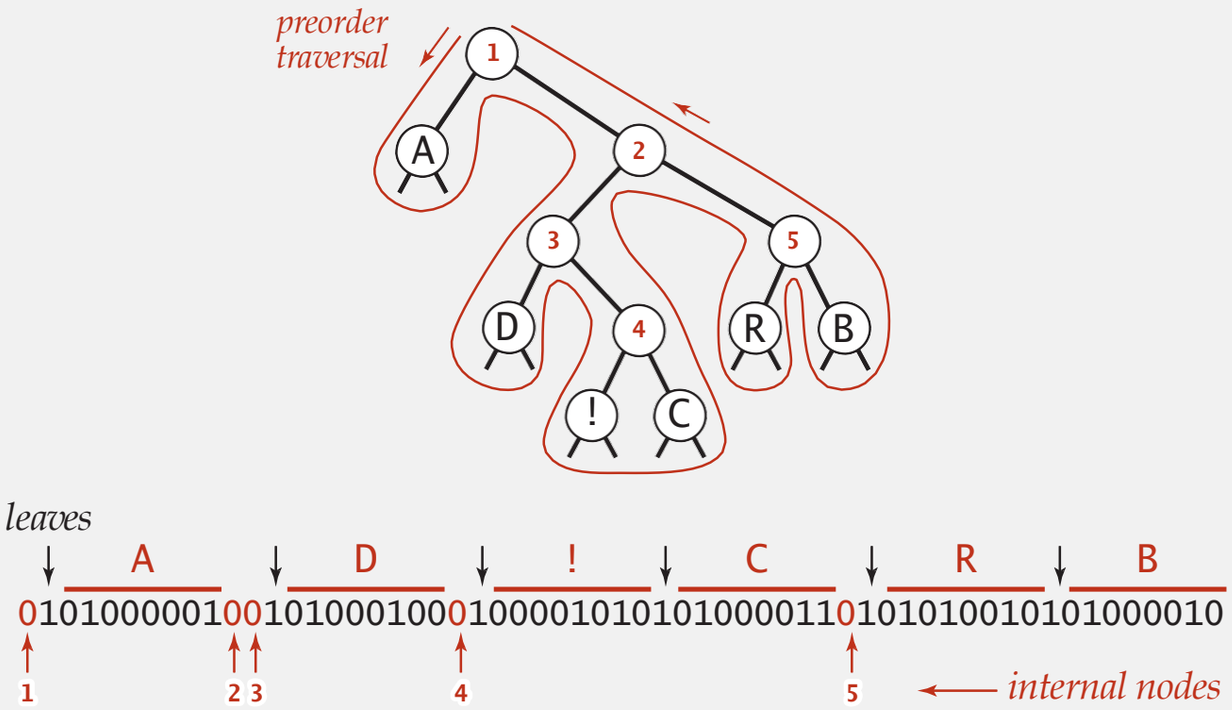
expand codeword for i^{th} char

Koha e ekzekutimit. Lineare në input me gjatësi N.

Prefix-free codes: si trasmetohet

Pyetje. Si të write një trie?

Përgjigje. Write preorder traversal i një trie; mark leaf dhe internal nodes me një bit.



Using preorder traversal to encode a trie as a bitstream

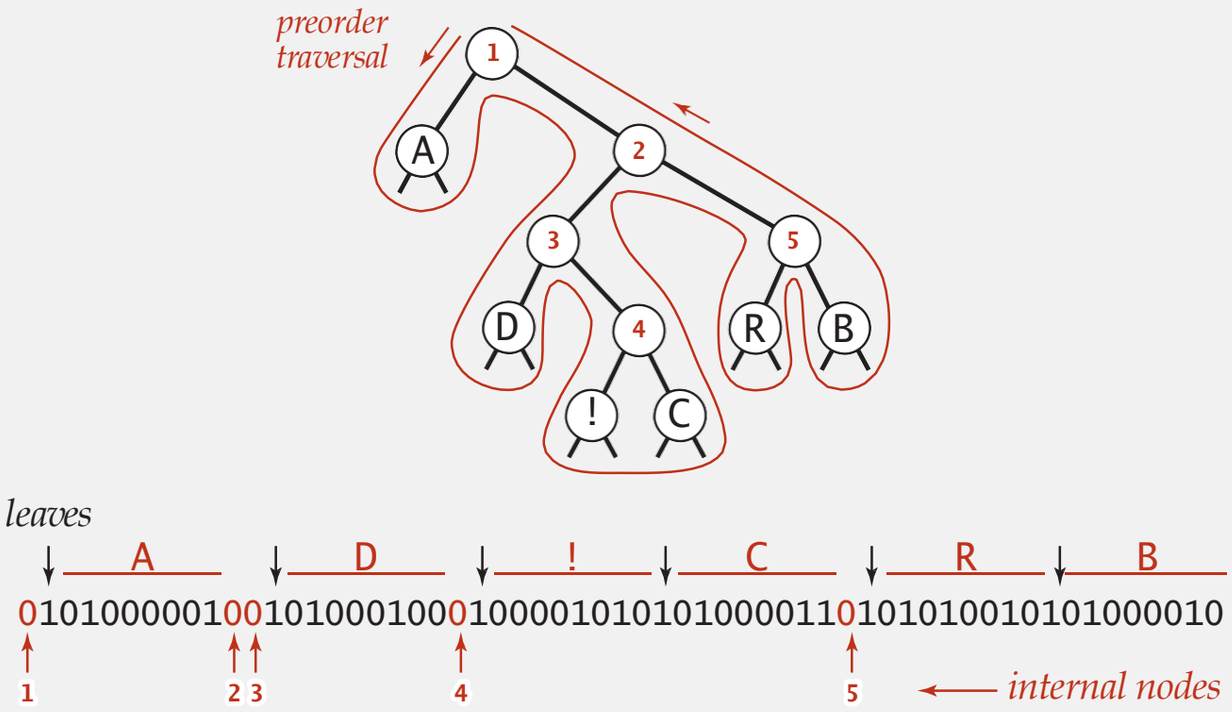
```
private static void writeTrie(Node x)
{
   if (x.isLeaf())
   {
      BinaryStdOut.write(true);
      BinaryStdOut.write(x.ch, 8);
      return;
   }
   BinaryStdOut.write(false);
   writeTrie(x.left);
   writeTrie(x.right);
}
```

Shënim. Nëse mesazhi është i gjatë, overhead i trie transmetues është i vogël.

Prefix-free codes: si trasmetohet

Pyetje. Si të read një trie?

Përgjigje. Rekonstruktohet prej preorder traversal të trie.



Përdorimi i preorder traversal për enkodim të trie si një bitstream

```
private static Node readTrie()
{
   if (BinaryStdIn.readBoolean())
   {
      char c = BinaryStdIn.readChar(8);
      return new Node(c, 0, null, null);
   }
   Node x = readTrie();
   Node y = readTrie();
   return new Node('\0', 0, x, y);
}
```

arbitrary value
(value not used with internal nodes)

Huffman algorithm demo

- Njehsohet frekuenca për çdo karakter në input.



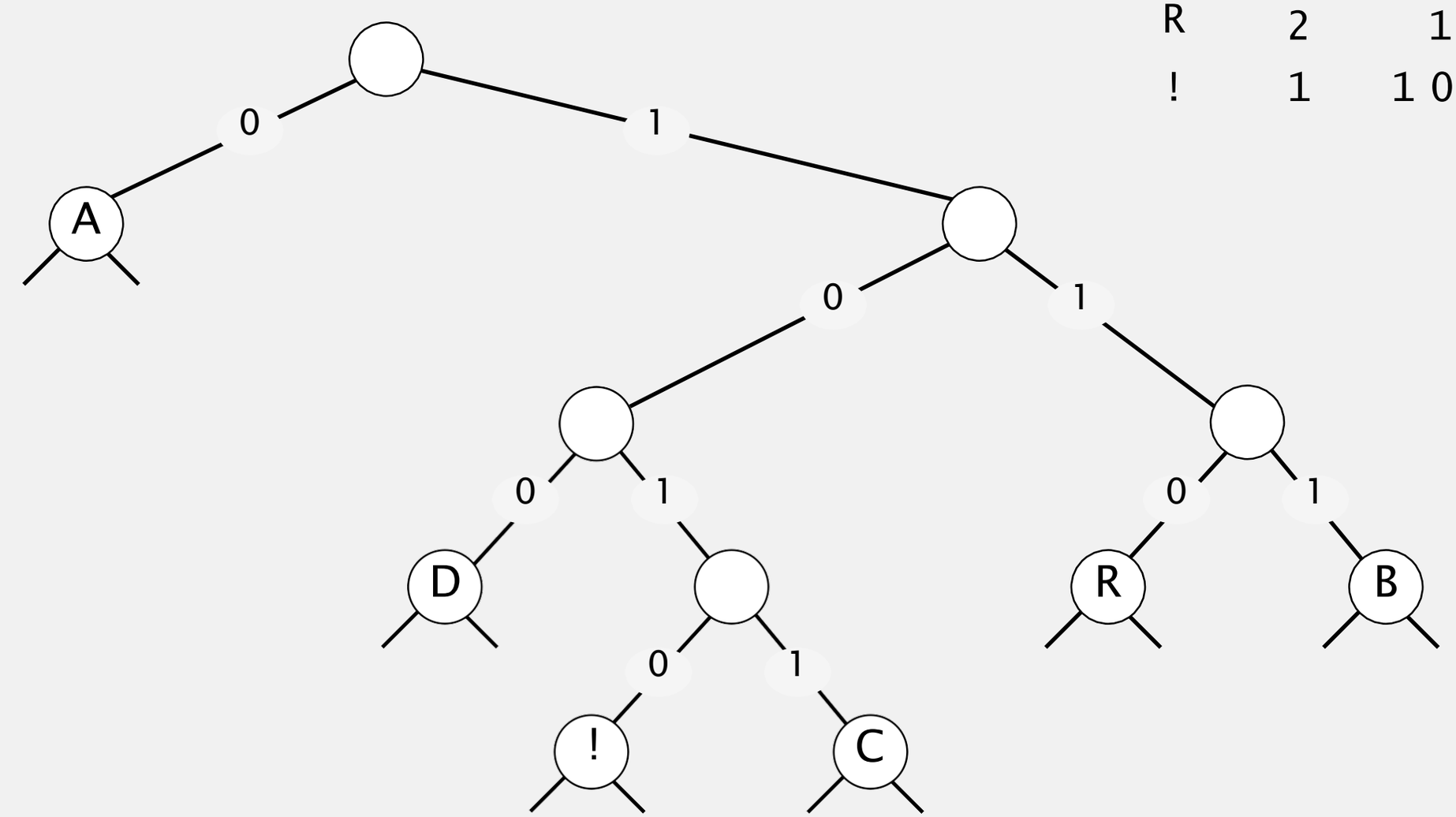
char	freq	encoding
A	5	
B	2	
C	1	
D	1	
R	2	
!	1	

input

A B R A C A D A B R A !

Huffman algorithm demo

char	freq	encoding
A	5	0
B	2	1 1 1
C	1	1 0 1 1
D	1	1 0 0
R	2	1 1 0
!	1	1 0 1 0



Huffman kodi

Pyetje. Si të gjindet prefix-free code më i mirë?

Huffman algorithm:

- Njehësohet frekuenca `freq[i]` për çdo char `i` në input.
- Starto me një node që korespondon me çdochar `i` (me peshë `freq[i]`).
- Përsëritet derisa të krijohet një trie:
  - Selektohen dy tries me min peshë `freq[i]` dhe `freq[j]`
  - bashkohen në një trie të vetëm me peshë `freq[i] + freq[j]`

Applications:











Konstruktimi i trie me Huffman trie: Java implementimi

```
private static Node buildTrie(int[] freq)
{
   MinPQ<Node> pq = new MinPQ<Node>();
   for (char i = 0; i < R; i++)
      if (freq[i] > 0)
         pq.insert(new Node(i, freq[i], null, null));

   while (pq.size() > 1)
   {
      Node x = pq.delMin();
      Node y = pq.delMin();
      Node parent = new Node('\0', x.freq + y.freq, x, y);
      pq.insert(parent);
   }

   return pq.delMin();
}
```

initialize PQ with singleton tries

merge two smallest tries

not used for internal nodes

total frequency

two subtries

Huffman enkodimi - përmbledhje

Teoremë. [Huffman 1950at] Algoritmi i Huffman-it jep një prefix-free code optimal

asnjë prefix-free code nuk pëdor më pak bita

Implementimi.

- Pass 1: tabelohën char frekuencat dhe ndërtohet trie.
- Pass 2: enkodohet file-i duke traverse trie ose lookup table.

Kohëzgjatja. Duke përdorë binary heap

$$N + R \log R .$$

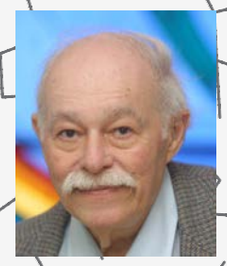
input i — alfabeti



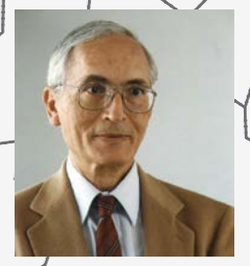
http://algs4.cs.princeton.edu

5.5 Data Compression

- *introduction*
- *run-length coding*
- *Huffman compression*
- *LZW compression*



Abraham Lempel



Jacob Ziv

Metodat statistikore

Modeli statik. Modeli i njejtë për të gjitha tekstet.

- Shpejtë.
- Nuk është optimal: Tekstet e ndryshme kanë veçori të ndryshme statistikore.
- P.sh.: ASCII, Morse code.

Modeli dinamik. Gjenerohet modeli bazuar në tekst.

- Një pass preliminar duhët për ta gjeneruar modelin.
- Modeli duhët të transmetohet.
- Shembull: Huffman code.

Modeli adaptiv. Modeli mësohet dhe update në mënyrë progresive përgjatë leximit të tesktit.

- Modelimi më i saktë jep kompresion më të mire.
- Dekodimi duhet të starton nga fillimi.
- P.sh.: LZW.

LZW compression demo

input	A	B	R	A	C	A	D	A	B	R	A		R		B			
matches	A	B	R	A	C	A	D	A	B		R	A		B	R		A	B	R			A	
vlera	41	42	52	41	43	41	44	81			83			82			88					41	80

LZW compression për A B R A C A D A B R A B R A B R A

key	value
⋮	⋮
A	41
B	42
C	43
D	44
⋮	⋮

key	value
AB	81
BR	82
RA	83
AC	84
CA	85
AD	86

key	value
DA	87
ABR	88
RAB	89
BRA	8A
ABRA	8B

codeword table

Lempel-Ziv-Welch compression

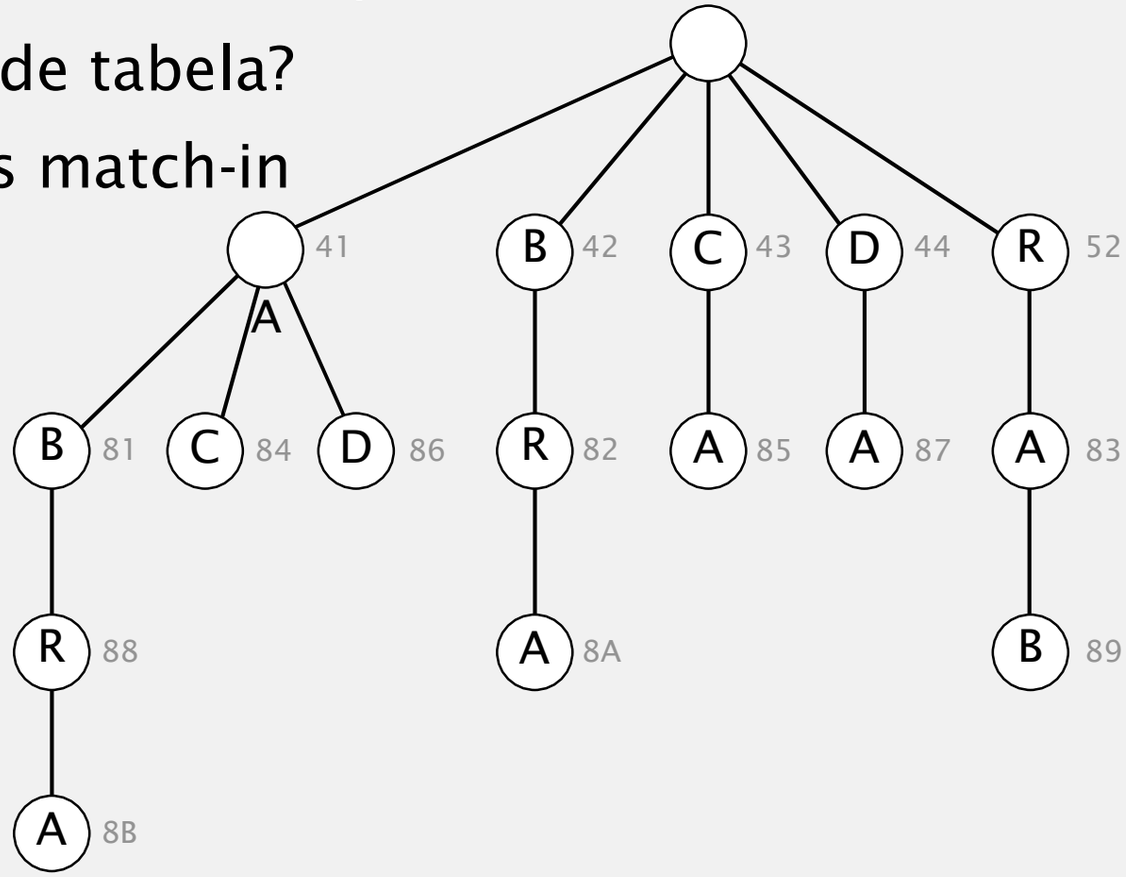
LZW compression.

- Krijohet një ST që lidhë W-bit codewords me string çelësa.
- Inicializohet ST me codewords për single-char çelësa.
- Gjindet stringu s më i gjatë në ST që është një prefiks i pjesës së paskenuar të inputit.
- Write W-bit codeword që lidhet me s.
- Add $s + c$ në ST, ku c është char I ardhshëm në input.

prefiksi match-i më i gjatë

Pyetje. Si paraqitet LZW compression code tabela?

Përgjigje. Një trie për të përkrahë prefiks match-in më të gjatë.



LZW zgjerimi - demo

value	41	42	52	41	43	41	44	81	83	82	88	41	80
output	A	B	R	A	C	A	D	A B	R A	B R	A B R	A	

LZW expansion for 41 42 52 41 43 41 44 81 83 82 88 41 80

çelësi	vlera
⋮	⋮
41	A
42	B
43	C
44	D
⋮	⋮

çelësi	vlera
81	AB
82	BR
83	RA
84	AC
85	CA
86	AD

çelësi	vlera
87	DA
88	ABR
89	RAB
8A	BRA
8B	ABRA

codeword table

LZW zgjerimi

LZW zgjerimi

- Krijohet ST që lidhë string vlerat me W-bit çelësa.
- Inicializohet ST që përmbanë single-char vlera.
- Read një W-bit çelës.
- Gjindet string vlera lidhëse në ST dhe shkruhet (write).
- Update ST.

Pyetje. Si paraqitet LZW code table për zgjerim?

Përgjigje. Një array me madhësi 2^W.

*ST = Symbol table

key	value
⋮	⋮
65	A
66	B
67	C
68	D
⋮	⋮
129	AB
130	BR
131	RA
132	AC
133	CA
134	AD
135	DA
136	ABR
137	RAB
138	BRA
139	ABRA
⋮	⋮

LZW – rast i ndëlikuar: kompresimi

input	A	B	A	B			A		
matches	A	B	A B		A B A				
vlera	41	42	81		83			80	

LZW kompresimi për ABABABA

çelësi	vlera
⋮	⋮
A	41
B	42
C	43
D	44
⋮	⋮

çelësi	vlera
AB	81
BA	82
ABA	83

codeword table

LZW – rast i ndëlikuar:zgjerimi

value	41	42	81	83	80
output	A	B	A B	A B A	

Duhet të dihet cili çelës ka vlerën 83 para se të jetë në ST!

LZW zgjerimi për 41 42 81 83 80

çelësi	vlera
⋮	⋮
41	A
42	B
43	C
44	D
⋮	⋮

çelësi	vlera
81	AB
82	BA
83	ABA

codeword table